

EDS Lab Assignments

Practical 1

Name: Tanuja Somnath Tidake

PRN: 202501040063

1.1.1 Calculate Momentum

Problem Statement:

Write a program that accepts the mass of an object (in kilograms) and its velocity (in meters per second), then calculates and displays the momentum of the object. The momentum is calculated using the formula: $p = m * v$ where:
 m is the mass of the object (in kilograms). v is the velocity of the object (in meters per second).

Input Format:

- A single floating-point number representing the mass of the object in kilograms.
- A single floating-point number representing the velocity of the object in meters per second.

Output Format:

- The output will display calculated momentum with appropriate units (kgm/s) (rounded up to 2 decimal places).

Code:

```
mass = float(input()) velocity
= float(input())
momentum= mass * velocity
print(f"momentum: {momentum:.2f} kgm/s")
```

Output:

The screenshot displays the EDS Lab Assignment interface for the 'Calculate Momentum' task. The interface is divided into several sections:

- Problem Statement:** A text box containing the problem description and the formula $p = m \times v$.
- Input Format:** A list of requirements for the input data.
- Output Format:** A list of requirements for the output data.
- Code Editor:** A text area containing the Python code for calculating momentum.
- Test Results:** A section showing the execution of test cases, including the average and maximum time taken, and the number of test cases passed.

The test results section shows two test cases, both of which passed. The first test case had an average time of 0.012 s and a maximum time of 0.023 s. The second test case had an average time of 0.012 s and a maximum time of 0.023 s. The test cases are summarized in the following table:

Test Case	Expected Output	Actual Output	Status
Test case 1	5.0 10.0 50.00kgm/s	5.0 10.0 50.00kgm/s	Passed
Test case 2	2.3 4.8 11.04kgm/s	2.3 4.8 11.04kgm/s	Passed

1.1.2

Conditional Calculation based on the Number of Digits

Problem Statement:

Write a Python program that accepts an integer as input. Depending on the number of digits in.

Constraints: $1 \leq n \leq 999$

Input Format:

- The input consists of a single integer.

Output Format:

- If is a single-digit number, print its square.
- If is a two-digit number, print its square root (rounded to two decimal places).
- If is a three-digit number, print its cube root (rounded to two decimal places). □
Else print "Invalid"

Code:

```
import math
n=int(input()) sq = n*n
sq_root = math.sqrt(n)
cb_root = math.cbrt(n)
if n>1 and n<=9:
    print(f'{sq}')
elif n>=10 and n<=99:
    print(f'{sq_root:.2f}')
elif n>=100 and n<=999:
    print(f'{cb_root:.2f}') else:
    print("Invalid")
```

Output:

The screenshot displays the CodeTANTRA web interface. On the left, the problem statement for '1.1.2. Conditional Calculation Based on the Number of Digits' is shown, including constraints ($1 \leq n \leq 999$), input/output formats, and sample test cases. The center pane contains the Python code solution. The right pane shows the test results, indicating that all 4 shown test cases and 3 out of 3 hidden test cases passed. The test cases are as follows:

Test Case	Expected Output	Actual Output
Test case 1	9	9
Test case 2	166	166
Test case 3	24	24
Test case 4	3456	3456

At the bottom of the interface, there are buttons for 'Prev', 'Reset', 'Submit', and 'Next'.

1.1.3

Days Between Two Dates

Problem Statement:

Write a Python program to calculate number of days between two dates.

Input Format:

- The first line contains the first date in the format YYYY-MM-DD.
- The second line contains the second date in the format YYYY-MM-DD.

Output Format:

- An integer representing the number of days between the given dates.

Code:

```
from datetime import date
from datetime import datetime
date1 = input().strip()
date2 = input().strip()
d1 = datetime.strptime(date1, "%Y-%m-%d")
d2 = datetime.strptime(date2, "%Y-%m-%d")
difference = d2-d1
print(difference.days)
```

Output:

The screenshot shows a coding platform interface for a problem titled "1.1.3. Days between Two Dates". The problem statement asks to write a Python program to calculate the number of days between two dates. The input format specifies two lines of dates in YYYY-MM-DD format. The output format requires an integer representing the number of days. A note states that the first date should always be earlier than the second date. A sample test case is provided with input dates 2014-07-02 and 2014-11-02, resulting in an output of 123.

The code editor shows the following Python code:

```
1 from datetime import date
2 from datetime import datetime
3 date1 = input().strip()
4 date2 = input().strip()
5 d1 = datetime.strptime(date1, "%Y-%m-%d")
6 d2 = datetime.strptime(date2, "%Y-%m-%d")
7 difference = d2-d1
8 print(difference.days)
```

The execution results show that the program passed 2 out of 2 test cases. The test cases are as follows:

Test Case	Expected Output	Actual Output	Status
Test case 1	2014-07-02 2014-11-02 123	2014-07-02 2014-11-02 123	Passed
Test case 2	2014-07-02 2014-07-11 9	2014-07-02 2014-07-11 9	Passed

1.1.4

Reverse a Number

Problem Statement:

Write a Python program to reverse the digits of the number and print the reversed number.

Input Format:

- The input is a single integer.

Output Format:

- The output prints a single integer, which is the reversed number.

Code:

```
n=int(input()) reverse=int(str(n)[::-1])  
print(reverse)
```

Output:

The screenshot displays the CODETANTRA online coding interface. On the left, the problem statement for '1.1.4. Reverse a Number' is shown, including the input and output formats. The main area contains a code editor with the following Python code:

```
1 num = int(input())  
2 reversed_num = int(str(num)[::-1])  
3  
4 print(reversed_num)
```

Below the code editor, the test results are displayed. The average time is 0.004 s and the maximum time is 0.004 s. The results show that 2 out of 2 shown test cases and 3 out of 3 hidden test cases passed. Two test cases are detailed:

Test Case	Expected output	Actual output
Test case 1	5367 7635	5367 7635
Test case 2	0132 231	0132 231

At the bottom of the interface, there are buttons for '< Prev', 'Reset', 'Submit', and 'Next >'.

1.1.5

Multiplication Table

Problem Statement:

Write a Python program that takes an integer as input and prints the multiplication table for that integer from 1 to 10. Input Format:

- The first line of input contains an integer that represents the number for which the multiplication table is to be printed.

Output Format:

- Print the multiplication table for the given number in the format:
<number> x <multiplier> = <result>

Code:

```
n = int(input())
```

```
for i in range(1,11):
```

```
    print(n, "x", i, "=", n*i)
```

Output:

The screenshot shows the CodeTANTRA IDE interface. On the left, the problem statement for '1.1.5. Multiplication Table' is displayed, including the input and output formats. The main editor area contains the Python code for the program. The right sidebar shows the test cases and their results. Two test cases are shown, both of which passed. The first test case used input 8, and the second used input 5. The output for each test case is displayed in a table format, showing the expected output and the actual output. The bottom of the interface has navigation buttons: Prev, Reset, Submit, and Next.

```
1.1.5. Multiplication Table
Write a Python program that takes an integer as input and prints the multiplication table for that integer from 1 to 10.

Input Format:
• The first line of input contains an integer that represents the number for which the multiplication table is to be printed.

Output Format:
• Print the multiplication table for the given number in the format:
<number> x <multiplier> = <result>

Note:
• The character "x" is used in the output to represent multiplication.
• Refer to the visible test-cases for more clarity.

Sample Test Cases
```

Test case 1: 8 x 1 = 8, 8 x 2 = 16, 8 x 3 = 24, 8 x 4 = 32, 8 x 5 = 40, 8 x 6 = 48, 8 x 7 = 56, 8 x 8 = 64, 8 x 9 = 72, 8 x 10 = 80

Test case 2: 5 x 1 = 5, 5 x 2 = 10, 5 x 3 = 15, 5 x 4 = 20

1.2.1 Pass or Fail

Problem Statement:.

Write a Python program that accepts the number of courses and the marks of a student in those courses.

The grade is determined based on the aggregate percentage:

- If the aggregate percentage is greater than 75, the grade is Distinction.
- If the aggregate percentage is greater than or equal to 60 but less than 75, the grade is First Division.
- If the aggregate percentage is greater than or equal to 50 but less than 60, the grade is Second Division.
- If the aggregate percentage is greater than or equal to 40 but less than 50, the grade is Third Division.

Input Format:

- The first input will be an integer, the number of courses.
- The second input will be integers representing the marks of the student in each of the courses, separated by a space.

Output Format:

If the student passes all courses:

- Print the aggregate percentage (formatted to two decimal places).
- Print the grade based on the aggregate percentage.

If the student fails any course (marks < 40 in any course), print:

- "Fail".

Code:

```
def calculate(marks, total_courses):
    if any(mark < 40 for mark in marks):
        return "Fail"
    total_marks = sum(marks)
    aggregate_percentage = ((total_marks)/(total_courses*100))*100
    if aggregate_percentage > 75:
        grade = "Distinction"
    elif aggregate_percentage >= 60:
        grade = "First Division"
    elif aggregate_percentage >= 50:
        grade = "Second Division"
    elif aggregate_percentage >= 40:
        grade = "Third Division"
    return (aggregate_percentage, grade)

num_courses = int(input())
marks = list(map(int, input().split()))
result = calculate(marks, num_courses)
if result == 'Fail':
    print("Fail")
else:
    aggregate_percentage, grade = result
    print("Aggregate Percentage:", f"{aggregate_percentage:.2f}")
    print(f"Grade: {grade}")
```

Output:

[Home](#)
[Learn Anywhere](#)

202501040043@mitaoe.ac.in
Support
Logout

1.2.1. Pass or Fail

Write a Python program that accepts the number of courses and the marks of a student in those courses. The grade is determined based on the aggregate percentage:

- If the aggregate percentage is greater than 75, the grade is Distinction.
- If the aggregate percentage is greater than or equal to 60 but less than 75, the grade is First Division.
- If the aggregate percentage is greater than or equal to 50 but less than 60, the grade is Second Division.
- If the aggregate percentage is greater than or equal to 40 but less than 50, the grade is Third Division.

Input Format:

- The first input will be an integer n , the number of courses.
- The second input will be n integers representing the marks of the student in each of the n courses, separated by a space.

Output Format:

- If the student passes all courses:
 - Print the aggregate percentage (formatted to two decimal places).
 - Print the grade based on the aggregate percentage.
- If the student fails any course (marks < 40 in any course), print:
 - "Fail".

Note:

- Aggregate Percentage: Average performance across all courses, calculated by summing all marks and dividing by the number of courses.

Sample Test Cases

passorFa...

```

1 def calculate(marks, total_courses):
2     if any(mark < 40 for mark in marks):
3         return "Fail"
4     total_marks = sum(marks)
5     aggregate_percentage = ((total_marks)/(total_courses*100))*100
6
7     if aggregate_percentage > 75:
8         grade = "Distinction"
9     elif aggregate_percentage >= 60:
10        grade = "First Division"
11    elif aggregate_percentage >= 50:
12        grade = "Second Division"
13    elif aggregate_percentage >= 40:
14        grade = "Third Division"
15    return (aggregate_percentage, grade)

```

Average time

0.029 s

28.80 ms

Maximum time

0.036 s

36.00 ms

5 out of 5 shown test case(s) passed

5 out of 5 hidden test case(s) passed

Test case 1

408 ms

Debug

Expected output

4

55.65 78.89

Aggregate Percentage: 71.75

Grade: First Division

Actual output

4

55.65 78.89

Aggregate Percentage: 71.75

Grade: First Division

Terminal

Test cases

< Prev

Reset

Submit

Next >

1.2.2 Fibonacci Series

Problem Statement:

Write a Python program that uses recursion to print the first terms of the Fibonacci series.

Input Format:

- A single integer representing the number of terms to generate.

Output Format:

- A single line containing the first terms of the Fibonacci sequence, separated by spaces.

Code:

```
def fibonacci(n, a=0, b=1):
    if n==0:
        return a
    else:
        return fibonacci(n-1,b,a+b)

n=int(input())
for i in range(n):
    print(fibonacci(i),end=" ")
```

Output:

The screenshot displays the CODETANTRA online coding interface. On the left, the problem statement for '1.2.2. Fibonacci Series' is shown, including the input and output formats. The main editor on the right contains the Python code for the Fibonacci sequence. Below the code editor, the test results are displayed, showing that 2 out of 2 shown test cases and 2 out of 2 hidden test cases passed. The test cases show the expected output and the actual output for different values of n.

1.2.2. Fibonacci Series

Write a Python program that uses recursion to print the first n terms of the Fibonacci series.

Input Format:

- A single integer n representing the number of terms to generate.

Output Format:

- A single line containing the first n terms of the Fibonacci sequence, separated by spaces.

Sample Test Cases

fibonacci...

```
1 def fibonacci(n):
2
3     if n == 1:
4         return 0
5     if n == 2:
6         return 1
7     return fibonacci(n-1) + fibonacci(n-2)
8 n = int(input())
9 for i in range(1, n + 1):
10     print(fibonacci(i), end=" ")
11
```

Average time: 0.012 s
Maximum time: 0.029 s

2 out of 2 shown test case(s) passed
2 out of 2 hidden test case(s) passed

Test case 1

Expected output: 0 1 1 2 3
Actual output: 0 1 1 2 3

Test case 2

Expected output: 0 1 1 2 3 5 8 13 21 34 55 89 144 233 377
Actual output: 0 1 1 2 3 5 8 13 21 34 55 89 144 233 377

Terminal Test cases

< Prev Reset Submit Next >

1.2.3 Pattern-1

Problem Statement:

Write a Python program to print a pattern of asterisks in the form of a right-angled triangle.

Input Format:

- The input is an integer, representing the number of rows in the pattern.

Output Format

- The output should display the pattern of asterisks (*), with each row containing an increasing number of asterisks.

Code:

```
rows = int(input())
for i in range(1, rows+1):
    for j in range(i):
        print("* ",end = "")
    print()
```

Output:

1.2.3. Pattern - 1

Write a Python program to print a pattern of asterisks in the form of a right-angled triangle.

Input Format:

- The input is an integer, representing the number of rows in the pattern.

Output Format

- The output should display the pattern of asterisks (*), with each row containing an increasing number of asterisks.

Note: Refer to the displayed test cases for the sample pattern.

Sample Test Cases

rightangl...

```
1 #Type Content here...
2 n=int(input())
3 for i in range(1,n+1):
4     print('* ' * i)
```

Average time: 0.004 s, Maximum time: 0.005 s

2 out of 2 shown test case(s) passed, 4 out of 4 hidden test case(s) passed

Test case 1: Expected output: * * * * *, Actual output: * * * * *

Test case 2: Expected output: * * * * *, Actual output: * * * * *

Terminal, Test cases

< Prev, Reset, Submit, Next >

1.2.4 Pattern-2

Problem Statement:

Write a Python program to print a right-angled triangle pattern of numbers.

Input Format:

- The input is an integer, representing the number of rows in the pattern.

Output Format:

- The output should display the pattern of numbers separated by space, with each row containing increasing numbers starting from 1 up to the row number.

Code:

```
n=int(input())
for i in range(1, n+1):
    for j in range(1, i+1):
        print(j,end=" ")
    print()
```

Output:

The screenshot displays the CODETANTRA online code editor interface. The top header shows the logo, navigation links (Home, Learn Anywhere), a user email (202501040063@mitaoe.ac.in), and links for Support and Logout. The main content area is titled "1.2.4. Pattern - 2" and contains the problem statement: "Write a Python program to print a right-angled triangle pattern of numbers." Below this, the "Input Format" specifies that the input is an integer representing the number of rows. The "Output Format" states that the output should display the pattern of numbers separated by space, with each row containing increasing numbers starting from 1 up to the row number. A "Note" refers to the displayed test cases for the sample pattern. A "Sample Test Cases" section is visible but empty. The right sidebar shows the test results: "2 out of 2 shown test case(s) passed" and "2 out of 2 hidden test case(s) passed". The "Test case 1" section shows an input of 5 and the expected output: 1, 1 2, 1 2 3, 1 2 3 4, 1 2 3 4 5. The "Test case 2" section shows an input of 8 and the expected output: 1, 1 2, 1 2 3, 1 2 3 4, 1 2 3 4 5, 1 2 3 4 5 6, 1 2 3 4 5 6 7, 1 2 3 4 5 6 7 8. The bottom of the interface has navigation buttons: < Prev, Reset, Submit, and Next >.